

제 9 주:

배열로 구현된 환경 큐



강 지 훈

jhkang@cnu.ac.kr

충남대학교 컴퓨터공학과



[제 9 주 실습]

배열로 구현된 큐
- 배열을 고리 형태로 -

□ 실습 목표

■ 이론적 관점

- 큐의 개념

■ 구현적 관점

- 환형 큐로 구현하는 방법

과제에서 해결할 문제



□ [프로그램 09]

- 키보드에서 문자를 반복 입력 받는다.
 - 한번에 한 개의 문자를 입력 받는다.

- 입력의 종료 조건
 - '!' 키를 치면 더 이상 입력을 받지 않는다.

- 매번 한 개의 문자를 입력 받을 때 마다, 문자에 따라 정해진 일을 한다.

이 과제에서 필요한 객체는?

- ApplicationController

- AppView

- Model

- Class "CircularArrayQueue<T>"
- Enum "MessageID"

□ 문자에 따라 해야 할 일 [1]

- 영문자 ('A' ~ 'Z', 'a' ~ 'z'): 큐에 삽입한다.
 - 다음과 같은 메시지를 내보낸다. (입력된 영문자가 'x' 라면)
 - ◆ "[EnQueue] 삽입된 원소는 'x' 입니다."
 - 만일 큐가 full 이면 다음과 같은 메시지를 내보낸다.
 - ◆ ERROR : 큐가 꽉 차서 삽입이 불가능합니다.

- '!': 큐의 원소를 모두 삭제하고, 프로그램을 종료한다.
 - "큐 입력을 종료합니다" 메시지를 내보낸다.
 - 큐를 모두 비운다.
 - 또한 다음과 같은 통계 자료를 출력한다.
 - ◆ ".....입력된 문자는 모두 16 개입니다."
 - ◆ ".....정상 처리된 문자는 15 개입니다."
 - ◆ ".....무시된 문자는 1 개입니다."
 - ◆ ".....삽입된 문자는 6 개입니다."

□ 문자에 따라 해야 할 일 [2]

- 숫자문자('0' ~ '9'): 해당 수 만큼 큐에서 삭제한다.
 - 먼저 다음 메시지를 내보낸다. ('3'이 입력되었다면)
 - ◆ [RemoveN] 3 개의 원소를 삭제하려고 합니다.
 - 매번 삭제될 때마다 다음과 같은 메시지를 내보낸다. (삭제된 원소가 'x' 라면)
 - ◆ "[DeQueue] 삭제된 원소는 'x' 입니다.
 - 삭제를 하려는데 큐가 empty 이면, 다음과 같은 메시지를 내보내고 삭제를 멈춘다.
 - ◆ "ERROR : 큐에 원소가 없습니다."

- '-': 큐의 front 원소를 삭제한다.
 - 다음과 같은 메시지를 내보낸다. (front 원소가 'r' 라면)
 - ◆ "[DeQueue] 삭제된 원소는 'r' 입니다.
 - 삭제를 하려는데 큐가 empty 이면, 다음과 같은 메시지를 내보내고 삭제를 멈춘다.
 - ◆ "ERROR : 큐에 원소가 없습니다."

□ 문자에 따라 해야 할 일 [3]

- '#': 큐의 길이를 다음과 같이 출력한다. (현재 큐에 5 개의 원소가 있다면)
 - "[Size] 큐에는 현재 5 개의 원소가 있습니다."
- '/': 큐의 내용을 front 부터 rear 까지 다음과 같이 차례로 출력한다.
 - "[Queue] <Front> A b c d E f <Rear>"
- '^': 큐의 front 원소의 값을 출력한다. 큐는 변하지 않는다. (큐의 front 원소가 'f' 라면)
 - "[Front] 맨 앞 원소는 'f' 입니다."
 - 큐가 empty 이면 다음과 같은 메시지를 내보낸다.
 - ◆ "ERROR : 큐에 원소가 없습니다."

□ 문자에 따라 해야 할 일 [4]

- 그 밖의 문자들: 다음과 같이 출력하고 무시한다.
 - "ERROR : 의미 없는 문자가 입력되었습니다."

출력의 예

> 프로그램을 시작합니다.
 [큐 입력을 시작합니다.]
 - 문자를 입력하십시오: -
 [Empty] 큐에 원소가 없습니다.
 - 문자를 입력하십시오: A
 [EnQueue] 삽입된 원소는 A 입니다.
 - 문자를 입력하십시오: x
 [EnQueue] 삽입된 원소는 x 입니다.
 - 문자를 입력하십시오: #
 [Size] 큐에는 현재 2개의 원소가 있습니다.
 - 문자를 입력하십시오: h
 [EnQueue] 삽입된 원소는 h 입니다.
 - 문자를 입력하십시오: /
 [Queue] <Front> A x h <Rear>
 - 문자를 입력하십시오: W
 [EnQueue] 삽입된 원소는 W 입니다.
 - 문자를 입력하십시오: Z
 ERROR : 큐가 꽉 차서 삽입이 불가능합니다.
 - 문자를 입력하십시오: -
 [DeQueue] 삭제된 원소는 A 입니다.
 - 문자를 입력하십시오: ^
 [Front] 맨 앞 원소는 x입니다.
 - 문자를 입력하십시오: /
 [Queue] <Front> x h W <Rear>

- 문자를 입력하십시오: 2
 [RemoveN] 2개의 원소를 삭제하려고 합니다.
 [DeQueue] 삭제된 원소는 x 입니다.
 [DeQueue] 삭제된 원소는 h 입니다.
 - 문자를 입력하십시오: 3
 [RemoveN] 3개의 원소를 삭제하려고 합니다.
 [DeQueue] 삭제된 원소는 W 입니다.
 [Empty] 큐에 원소가 없습니다.
 [Empty] 큐에 원소가 없습니다.
 - 문자를 입력하십시오: B
 [EnQueue] 삽입된 원소는 B 입니다.
 - 문자를 입력하십시오: =
 ERROR : 의미 없는 문자가 입력 되었습니다.
 - 문자를 입력하십시오: e
 [EnQueue] 삽입된 원소는 e 입니다.
 - 문자를 입력하십시오: /
 [Queue] <Front> B e <Rear>
 - 문자를 입력하십시오: !
 [큐 입력을 종료합니다.]
 [DeQueue] 삭제된 원소는 B 입니다.
 [DeQueue] 삭제된 원소는 e 입니다.
입력된 문자는 모두 17 개입니다.
정상 처리된 문자는 16 개입니다.
무시된 문자는 1 개입니다.
삽입된 문자는 6 개입니다.

>프로그램을 종료합니다.

main()



□ main()을 위한 class

```
public class DS1_09_학번_이름 {  
    public static void main (String[] args)  
    {  
        ApplicationController appController = new ApplicationController() ;  
        // ApplicationController 가 실질적인 main class 이다.  
        appController.run() ;  
        // 여기 main()에서는 앱 실행이 시작되도록 해주는 일이 전부이다.  
    }  
}
```

Class "AppController"



□ ApplicationController: 인스턴스 변수, 생성자

```
public class ApplicationController {  
    // 비공개 변수들  
    private AppView _appView ;  
    private CircularArrayQueue<Character> _queue ;  
    private int _inputChars ; // 입력된 문자의 개수  
    private int _ignoredChars ; // 무시된 문자의 개수  
    private int _addedChars ; // 삽입된 문자의 개수  
  
    // 생성자  
    public ApplicationController()  
    {  
        this._appView = new AppView() ;  
        this.initCharCounts();  
    }  
  
    // 비공개함수의 구현  
    .....  
  
    // 공개함수의 구현  
    .....  
} // End of class "AppController"
```

□ ApplicationController: 비공개 함수 [1]

■ private void showFrontElement()

- this._queue 의 "frontElement()" 를 이용하여 front 원소를 얻는다.
- 얻어온 원소를 this._appView 의 "outputFrontElement()" 를 이용하여 출력한다.

■ private void showQueueSize()

- this._queue의 "size()" 를 이용하여 원소의 개수를 얻는다.
- 얻어온 원소를 this._appView 의 "outputQueueSize()" 를 이용하여 출력한다.

□ ApplicationController: 비공개 함수 [2]

■ private void showAll()

- this._queue 의 "elementAt()" 를 이용하여 front 원소부터 rear 원소까지 큐에 들어 있는 모든 원소를 차례대로 출력한다.

```
private void showAll () {  
    int queueSize = this._queue.size() ;  
    Character elementForOutput = null ;  
  
    this._appView.outputMessage ("[Queue] <Front>") ;  
    for ( int order = 0 ; order < queueSize ; order++ ) {  
        Character element = this._queue.elementAt (order) ;  
        this._appView.outputElement (elementForOutput) ;  
    }  
    this._appView.outputMessage (" <Rear>") ;  
}
```

□ ApplicationController: 비공개 함수 [3]

- private void initCharCounts()
 - this._inputChars, this._addedChars, this._ignoredChars을 초기화
- private void countAdded()
 - this._addedChars 를 1 증가
- private void countIgnored()
 - this._ignoredChars 를 1 증가
- private void countInputChar()
 - this._inputChars 를 1 증가

□ ApplicationController: 비공개 함수 [4]

■ private void add (Character anElement)

- 큐가 가득 찬 경우
 - ◆ MessageID.Error_InputFull 을 출력
- 정상적으로 enqueue가 되었을 경우
 - ◆ this._appView의 "outputAdd()" 를 이용하여 삽입된 원소를 출력한다.
 - ◆ this.countAdded() 를 호출하여 삽입된 원소의 숫자를 증가시킨다.
- 유의:
 - ◆ 삽입하는 원소의 자료형이 기본 자료형인 "char" 가 아니라, class "Character" 이다.

□ ApplicationController: 비공개 함수 [4]

- `private void removeOne ()`
 - `deQueue`을 실행하여 삭제된 원소를 얻어옴
 - 얻어온 원소를 `this._appView`의 `"outputRemove()"` 를 이용하여 출력한다.

- `private void removeN (int numberOfCharsToBeRemoved)`
 - `this._appView`의 `"outputRemoveN()"` 을 이용하여 삭제되는 원소의 개수를 출력한다.
 - `"removeOne()"` 을 삭제하는 횟수만큼 실행한다.

□ ApplicationController: 비공개 함수 [5]

■ private void conclusion()

- 큐에 들어있는 모든 원소를 deQueue 한다.
- 입력된 문자의 개수를 출력한다.
- 정상 처리된 문자의 개수를 출력한다.
- 무시된 문자의 개수를 출력한다.
- 삽입된 문자의 개수를 출력한다.

■ private void showMessage (MessageID aMessageID)

- 각 MessageID 에 맞는 String 을 찾아, this._appView의 "outputMessage()" 를 이용하여 출력한다.

□ ApplicationController: 유일한 공개 함수 "run()" [1]

```
public void run() {  
    this._queue = new CircularArrayQueue<Character> ();  
    char inputChar ;  
  
    this.showMessage(MessageID.Notice_StartProgram) ;  
    this.showMessage(MessageID.Notice_StartMenu) ;  
    inputChar = this._appView.inputCharacter() ;  
    while ( inputChar != '!' ) {  
        this.countInputChar () ;  
        if ( (inputChar >= 'A' && inputChar <= 'Z') ||  
            (inputChar >= 'a' && inputChar <= 'z') )  
        {  
            this.add ( Character.valueOf (inputChar) ) ;  
            // 또는, 간단히 다음과 같이:  
            // this.add (inputChar) ; // 이렇게 해도 오류 없이 처리됨  
        }  
        else if ( inputChar >= '0' && inputChar <= '9' ) {  
            this.removeN (Integer.parseInt (String.valueOf(inputChar))) ;  
        }  
        else if ( inputChar == '-' ) {  
            this.removeOne() ;  
        }  
    }  
}
```

□ ApplicationController: 유일한 공개 함수 "run()" [2]

```

public void run() {
    ..... // 앞 쪽의 내용 생략

    while ( inputElement != '!' ) {
        ..... // 앞 쪽의 내용 생략

        else if ( inputChar == '#' ) {
            this.showQueueSize() ;
        }
        else if ( inputChar == '/' ) {
            this.showAll() ;
        }
        else if ( inputChar == '^' ) {
            this.showFrontElement() ;
        }
        else {
            this.showMessage (MessageID.Error_WrongMenu) ;
            this.countIgnored () ;
        }
        inputChar = this._appView.inputCharacter () ;
    }
    this.showMessage (MessageID.Notice_EndMenu) ;
    this.conclusion() ;
    this.showMessage (MessageID.Notice_EndProgram) ;
}

```

Class "AppView"



□ AppView: 인스턴스 변수, 생성자

```
public class AppView {  
    // 비공개 상수/변수들  
    private Scanner _scanner ;  
  
    // 생성자  
    public AppView ()  
    {  
        this._scanner = new Scanner(System.in) ;  
    }  
  
    // 공개함수의 구현  
    .....  
}
```

□ Class "AppView" 의 공개함수

- 입력을 위한 공개 함수
 - public String inputString()
 - public char inputCharacter()
 - ◆ [실습 7]과 동일

□ Class "AppView" 의 공개함수

■ 출력을 위한 공개 함수

- public void outputAdd (char anElement)
- public void outputElement (char anElement)
- public void outputFrontElement (char anElement)
- public void outputQueueSize (int aQueueSize)
- public void outputRemove (char anElement)
- public void outputRemoveN
(int numberOfCharsToBeRemoved)

각 기능에 맞추어 구현 할 것

□ Class "AppView" 의 공개함수

■ 출력을 위한 공개 함수

- public void outputResult (
 int aNumberOfInputChars,
 int aNumberOfIgnoredChars,
 int aNumberOfAddedChars)
 - ◆ 입력된 문자 개수 출력
 - ◆ 정상 처리된 문자 개수 출력
 - ◆ 무시된 문자 개수 출력
 - ◆ 삽입된 문자 개수 출력
- public void outputMessage (String aMessage)

Enum "MessageID"



Enum "MessageID"

```
public enum MessageID {  
  
    // Message IDs for Notices:  
    Notice_StartProgram, Notice_EndProgram,  
    Notice_StartMenu,  
    Notice_EndMenu,  
  
    // Message IDs for Errors:  
    Error_WrongMenu,  
    Error_InputFull,  
    Error_Empty,  
  
} //End of enum "MessageID"
```

Class "CircularArrayQueue"



❑ CircularArrayQueue: 상수, 인스턴스 변수

```
public class CircularArrayQueue<T> {  
  
    // Constants  
    private static final int DEFAULT_CAPACITY = 100 ;  
  
    // Instance Variables  
    private int    _capacity ;  
    private int    _front ;  
    private int    _rear ;  
    private T[]    _elements ;  
}
```


□ CircularArrayQueue: 생성자

- `public CircularArrayQueue (int givenCapacity)`
 - `this._capacity` 를 `givenCapacity` 로 초기화
 - `this._front` 와 `this._rear` 는 0 으로 초기화
 - `this._elements` 배열의 원소의 자료형이 generic type 인 `T` 로 선언 되어 있다. 그러므로, 배열 원소의 자료형은 `Object` 로 해서 배열을 생성해야 한다.
 - ◆ `this._elements = (T[ ]) new Object[this._capacity] ;`
- `public CircularArrayQueue ()`
 - 기본 생성자
 - 위에 주어진 생성자와 동일한 초기화 기능을 한다.
 - ◆ 단, `this._capacity` 의 값은 `DEFAULT_CAPACITY` 로 초기화한다.

□ CircularArrayQueue: 공개함수 [1]

- public int capacity()
 - this._capacity 값을 반환
- public boolean isEmpty ()
 - this._front 와 this._rear 가 같은지 확인하여 같으면 true 를 반환
- public boolean isFull ()
 - 다음 삽입될 위치가 _front 와 같은지 확인하여 같으면 true 를 반환
 - 다음 삽입될 위치는 어떻게 확인 할까?
- public int size ()
 - 크기를 유지하는 인스턴스 변수가 존재하지 않는다. 계산을 하여 얻는다.
 - ◆ 인스턴스 변수가 존재하지 않는 getter 이다.
 - 계산 방법:
 - ◆ this._front 가 this._rear 보다 작거나 같으면 this._rear 에서 this._front 를 뺀 값을 반환
 - ◆ 클 경우 this._rear 에 this._capacity 를 더해서 this._front 를 뺀 값을 반환

□ CircularArrayQueue: 공개함수 [2]

- public T frontElement ()
 - 비어 있는 경우 null 을 반환
 - 비어 있지 않은 경우 가장 앞에 있는 값을 반환

- public boolean enqueue (T anElement)
 - 가득 차 있을 경우 false 를 반환
 - 가득 차 있지 않을 경우
 - ◆ this._rear 를 새로운 위치로 변경한다.
 - ◆ this._elements[] 의 this._rear 위치에 anElement 를 삽입

□ CircularArrayQueue: 공개함수 [3]

■ public T deQueue ()

- 비어있는 경우 null 을 반환
- 만약 비어 있지 않으면
 - ◆ this._front 의 값을 (+1 하여) 다음 위치로 조정
 - this._front 는 항상 front 원소의 직전 위치를 가리킨다.
 - ◆ this._elements[this._front] 의 원소, 즉 front 원소를 빼내어 보관해 놓는다.
 - ◆ front 원소를 배열로부터 제거한다.
 - 배열의 해당 칸 this._elements[this._front] 를 null 값으로 설정한다.
- 보관해 놓은 front 원소를 결과값으로 돌려준다.

□ CircularArrayQueue: 공개함수 [4]

■ public void clear ()

- this._front 와 this._rear 를 0 으로 초기화한다.
- this._elements[] 의 모든 칸을 null 로 초기화한다.

■ public T elementAt (int anOrder)

- 리스트에서 anOrder 위치의 원소를 반환한다.
- front 원소의 위치가 리스트의 맨 앞 위치, 즉 0 번째 위치이다.
 - ◆ $(\text{this._front} + 1) \% \text{this._capacity}$ 위치가 anOrder 이 0 인 위치이다.
 - ◆ 그러므로 $(\text{this._front} + 1 + \text{anOrder}) \% \text{this._capacity}$ 의 위치가 배열 this._elements[] 에서의 위치가 된다.

요약

□ [문제 9] 요약

■ 환형 큐

- 큐의 이해
- 환형 큐로 구현하는 이유?
- 구현 방법

■ 환형 큐는, 원소를 삽입하거나 삭제할 때에 front와 rear의 위치를 계산하여야 한다.

- 보고서에 본인이 구현한 **위치 계산 법**을 설명하시오.

보고서 작성



□ PDF 보고서 작성 방법

■ 겉장

- 제목: 자료구조 실습 보고서
- 부제목: [제09주] 배열로 구현된 한형 큐
- 제출일
- 학번/이름

■ 내용

1. 프로그램 설명서

1. 주요 알고리즘 /자료구조 /기타
2. 함수 설명서
3. 종합 설명서

2. 프로그램 장단점 분석

3. 실행 결과 분석

1. 입력과 출력 (화면 capture 시킬 것)
2. 결과 분석

4. 소스 코드

□ 과제 제출

■ 파일 이름 작명 방법

● DS1_09_학번_이름.zip

● 폴더의 구성

◆ DS1_09_학번_이름

■ 프로그램

– 프로젝트 폴더 / 소스

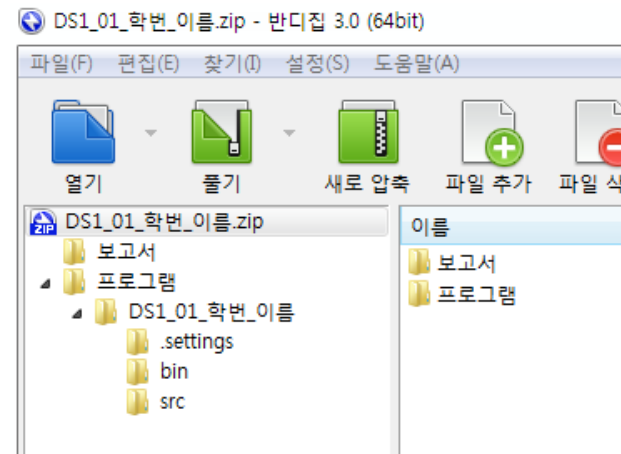
– 메인 클래스 이름: DS1_09_학번_이름.java

■ 보고서

– 이곳에 보고서 문서 파일을 저장한다.

– 입력과 실행 결과는 화면 image 로 문서에 포함시킨다.

– 문서는 pdf 파일로 만들어 제출한다.



[제 9 주] 실습 끝



